# Tópicos Avançados em Novas Tecnologias

## Automação e CI/CD

Prof. Diego Cardoso Borda Castro

# Pipelines

- Automação no mundo Devops
  - Acabar com processos repetitivos
  - Garantia de entrega de qualidade
  - Produtividade
  - Ganho de tempo
  - Revisar os processos
    - O processo deve estar correto
      - Antes de executar um processo automatizado
        - Execute ele manualmente

# Pipelines

- Uma tubulação / encanamento
  - Gerenciamento de fluxo de “Líquidos / produtos”
- Controle de fluxo de um processo
  - Automatizar e gerenciar um fluxo de trabalho
    - Entrada e saída de recursos
  - Renovar um certificado
  - Rotina de teste
  - Criação de um ambiente
  - …



# Pipelines

- Pipeline de CI (Integração contínua)
  - Integração de código novo
  - Colocar um código novo em um projeto já existente
    - Gerar uma nova versão
    - Uma nova release
      - DDL, JAR, imagem docker, …



# Pipelines

- Pipeline de CI (Integração contínua)
  - Exemplo básico
    - Necessariamente, irei criar com tudo isso?
      - Boas práticas
      - Não, depende da empresa
      - Não precisa ter tudo, precisamos começar



# Pipelines


INTEGRAÇÃO CONTÍNUA DO MARIO
BUILD
TESTES UNITÁRIOS
TESTES DE INTEGRAÇÃO
ANÁLISE DE CÓDIGO
CONSTRUÇÃO DO PACOTE

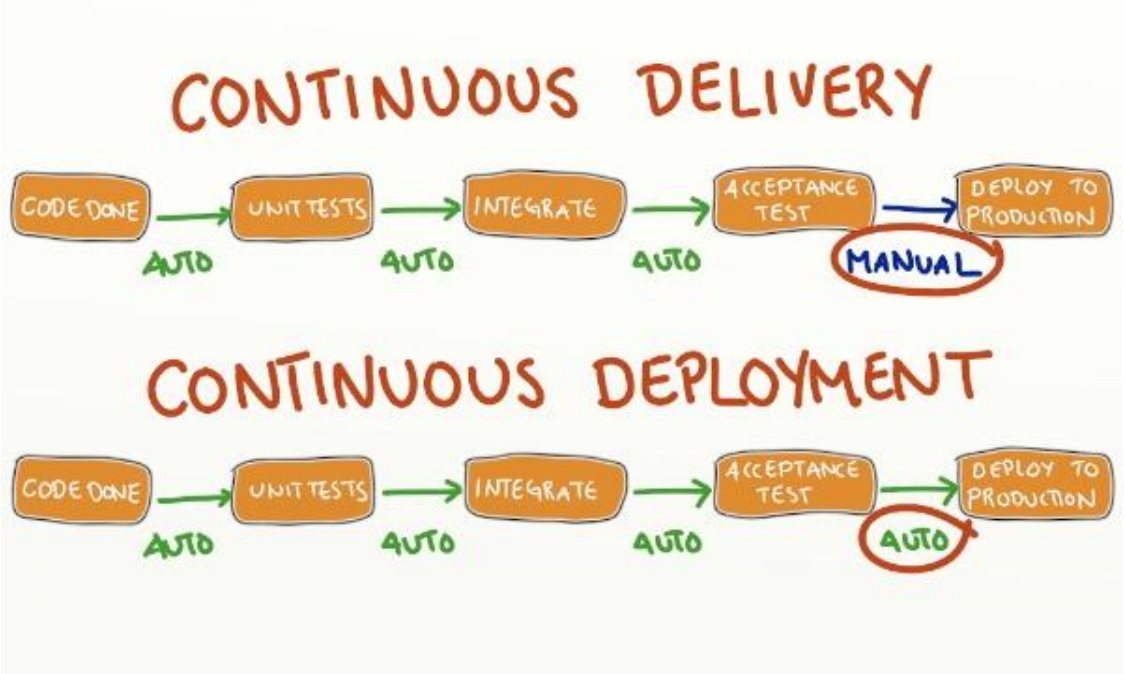
# Pipelines

- Pipeline de CD (Entrega contínua)
  - Continuous delivery or deployment
  - Vem logo depois da CI
  - Garante que o pacote que vem de CI seja deployado no ambiente de destino



# Pipelines

- Pipeline de CD
  - A Aprovação deve ser simples
  - Comece por ambiente de menor importância



# Pipelines


Entrega Contínua
Release
Deploy em Ambiente
Teste Funcional
Teste Manual
Aprovação

# Ferramentas de automação

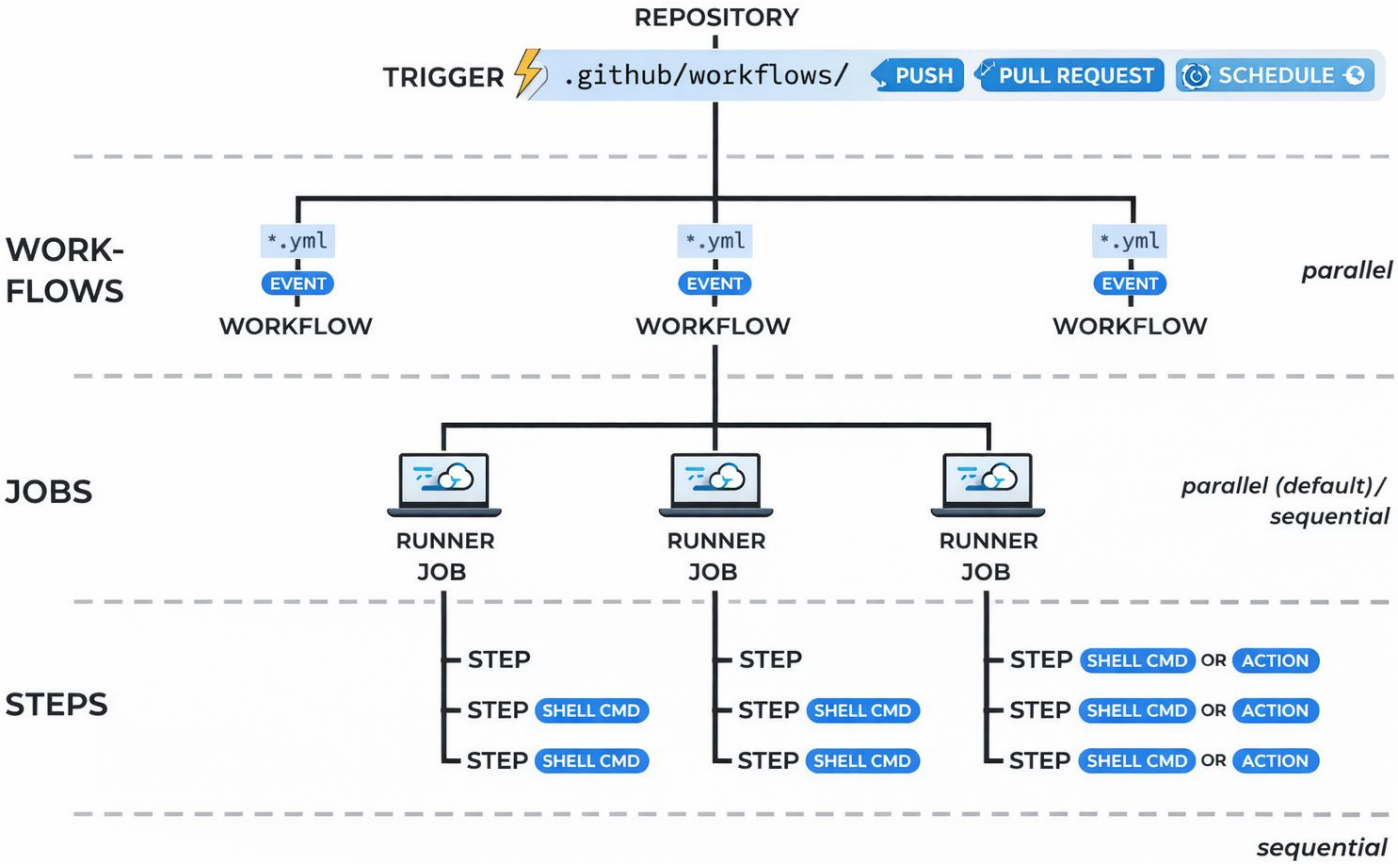
- CI/CD
  - Gitlab
  - Azure pipeline
  - Jenkins
  - CircleCI
  - Bitbucket Pipelines
  - AWS CodePipeline
  - Google Cloud Build
  - GitHub Actions
- Vantagens
  - Integração nativa com o GitHub
  - Fácil de configurar (YAML simples)
  - Não precisa de servidor próprio
  - Gratuito para projetos pequenos
  - Marketplace enorme de Actions prontas

# GitHub Actions

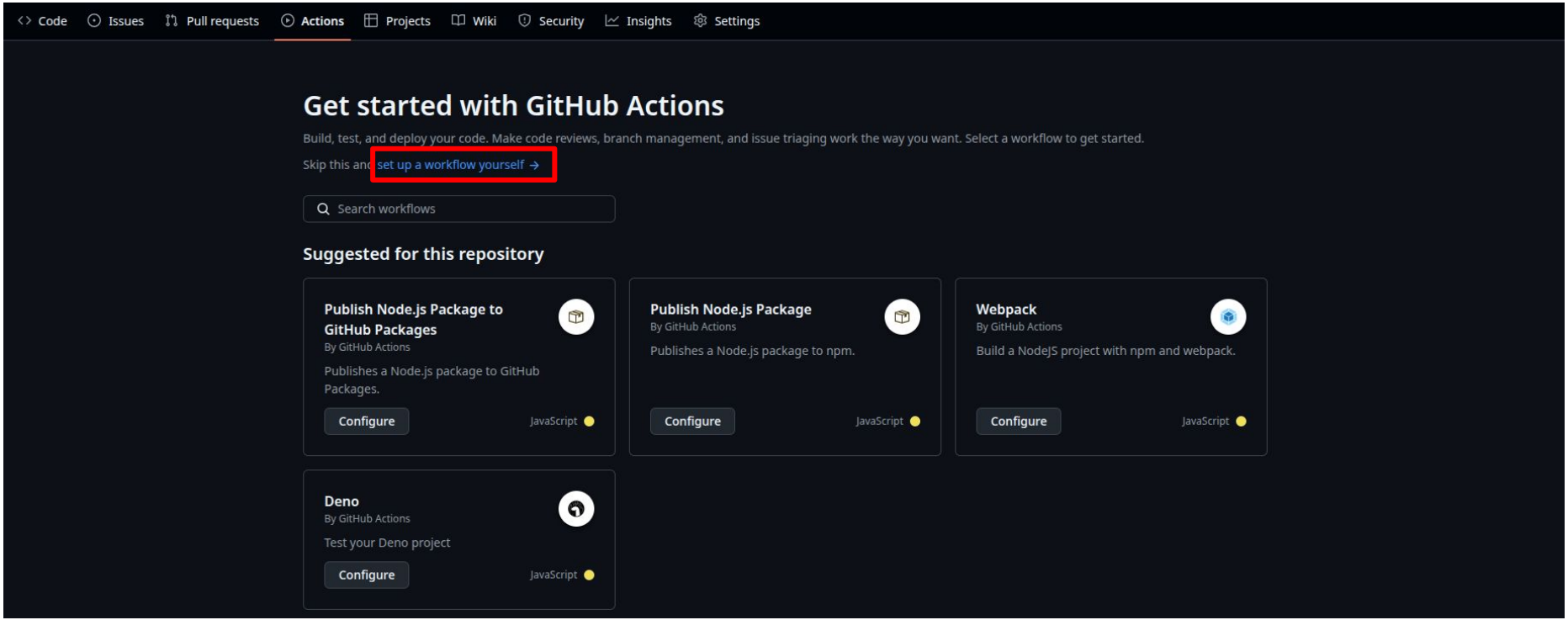
- Conceitos
  - Workflow
    - Declaração de automação
    - Em outras palavras:
      - Pipeline de processos
  - Trigger
    - Evento que vai disparar a automação
      - Exemplo: git em uma branch
  - Job
    - Coleção de tarefas que vai finalizar uma rotina
    - Job de teste, job de build

# GitHub Actions

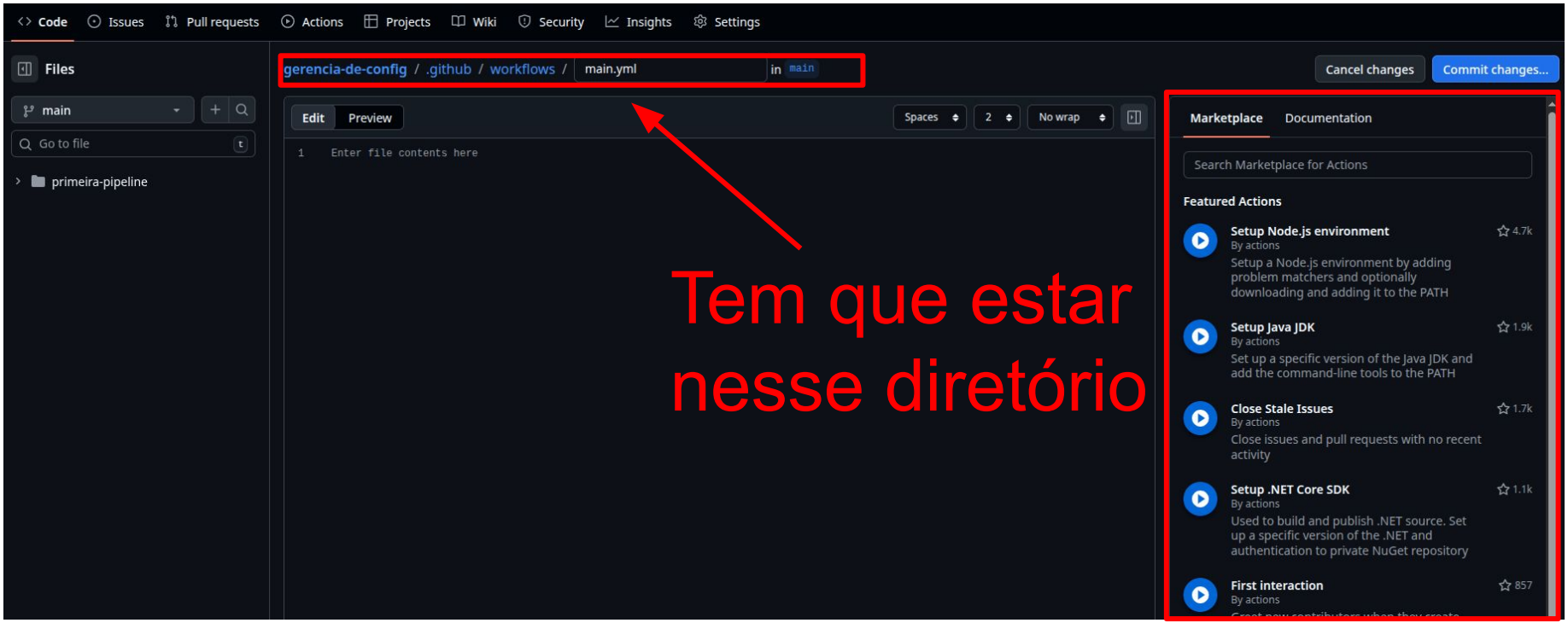
- Conceitos
  - Runner
    - O agente que vai executar o meu job
    - Basicamente a máquina que vai executar
  - Steps
    - Passos de execução de um job
  - Actions
    - Atividades que serão executadas dentro de um step


REPOSITORY
TRIGGER
.github/workflows/
PUSH
PULL REQUEST
SCHEDULE
WORK-FLOWS
*.yml
EVENT
WORKFLOW
*.yml
EVENT
WORKFLOW
*.yml
EVENT
WORKFLOW
parallel
JOBS
RUNNER JOB
RUNNER JOB
RUNNER JOB
parallel (default)/ sequential
STEPS
STEP
STEP SHELL CMD
STEP SHELL CMD
STEP
STEP SHELL CMD
STEP SHELL CMD
STEP SHELL CMD OR ACTION
STEP SHELL CMD OR ACTION
STEP SHELL CMD OR ACTION
sequential

# GitHub Actions



# GitHub Actions



# GitHub Actions [Sintaxe]

- Name
  - Define o nome do workflow
  - Aparece na aba Actions
- Run-name
  - Define o nome de cada execução
  - Cada vez que o workflow roda, pode ter um nome diferente
  - Pode usar variáveis (github, inputs, etc.)
    - run-name: Build by ${{ github.actor }}

# GitHub Actions [Sintaxe]

- ON
  - Define a trigger de execução
  - Evento que chama o workflow
    - Type
      - Tipo de atividade que vai chamar o workflow
    - Branches
      - Filtra a branch de execução

```yaml
on:
  pull_request:
    types:
      - opened
    branches:
      - 'releases/**'
```

```yaml
on:
  push:
    # Sequence of patterns matche
    branches:
      - main
      - 'mona/octocat'
      - 'releases/**'
    # Sequence of patterns matche
    tags:
      - v2
      - v1.*
```

# GitHub Actions [Sintaxe]

- ON
  - paths
    - Só executa se certos arquivos forem alterados
  - paths-ignore
    - NÃO roda se apenas esses arquivos mudarem

```yaml
on:
  push:
    paths:
      - 'sub-project/**'
      - '!sub-project/docs/**'
```

```yaml
on:
  push:
    paths-ignore:
      - 'docs/**'
```

# GitHub Actions [Sintaxe]

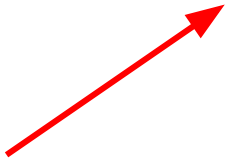
- ON
  - workflow_dispatch
    - Serve para executar o workflow manualmente pela aba Actions

```yaml
on:
  workflow_dispatch:
    inputs:
      logLevel:
        description: 'Log level'
        required: true
        default: 'warning'
        type: choice
        options:
          - info
          - warning
          - debug
      print_tags:
        description: 'True to print to STDOUT'
        required: true
        type: boolean
      tags:
        description: 'Test scenario tags'
        required: true
        type: string
      environment:
        description: 'Environment to run tests against'
        type: environment
        required: true
```

# GitHub Actions [Sintaxe]

- ON
  - workflow_run
    - Criar pipelines encadeadas
    - Um workflow execute depois do outro

```yaml
name: Build
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo build
```



```yaml
name: Deploy

on:
  workflow_run:
    workflows: ["Build"]
    types:
      - completed

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: echo deploy
```

# GitHub Actions [Sintaxe]

- ON
  - workflow_call
    - Serve para um workflow chamar outro workflow
      - Reutilizar pipeline
      - CI corporativo
      - monorepo
      - templates
      - …

```yaml
name: Reusable workflow example

on:
  workflow_call:
    inputs:
      config-path:
        required: true
        type: string
    secrets:
      token:
        required: true

jobs:
  triage:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/labeler@v6
      with:
        repo-token: ${{ secrets.token }}
        configuration-path: ${{ inputs.config-path }}
```

# GitHub Actions [Sintaxe]

- ON
  - workflow_call
    - Serve para um workflow chamar outro workflow
      - Reutilizar pipeline
      - CI corporativo
      - monorepo
      - templates
      - …

```yaml
name: Call a reusable workflow

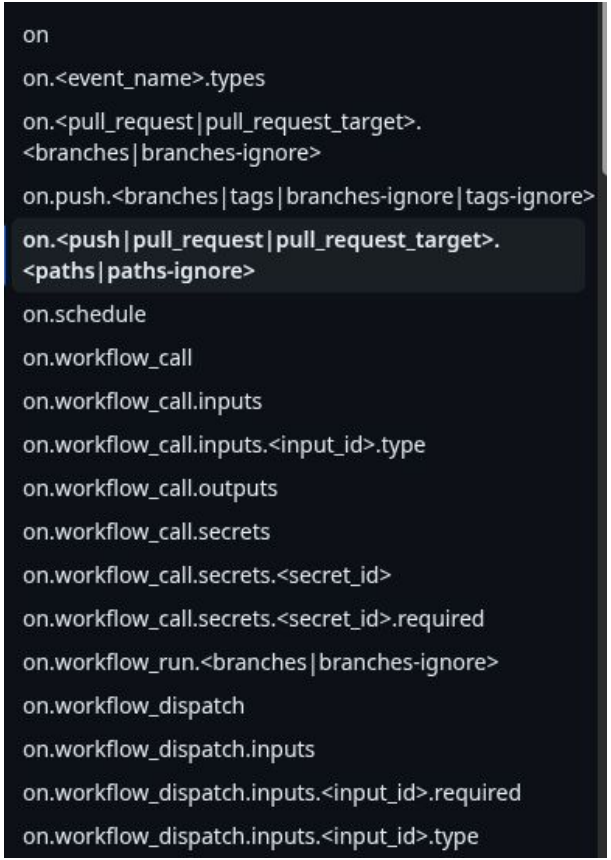
on:
  pull_request:
    branches:
      - main

jobs:
  call-workflow-passing-data:
    permissions:
      contents: read
      pull-requests: write
    uses: octo-org/example-repo/.github/workflows/workflow-B.yml@main
    with:
      config-path: .github/labeler.yml
    secrets:
      token: ${{ secrets.GITHUB_TOKEN }}
```

# GitHub Actions
# [Sintaxe]

- Existem mais de 50 eventos ON
  - push, pull_request, workflow_dispatch, schedule, release, workflow_call …

https://docs.github.com/pt/actions/reference/workflows-and-actions/events-that-trigger-workflows



# GitHub Actions [Sintaxe]

- Jobs
  - Sequência de tarefas chamadas etapas
  - Agrupamento de atividade

Sequencial

```yaml
jobs:
  build:
    runs-on: ubuntu-latest

  deploy:
    runs-on: ubuntu-latest
    needs: build
```

Paralelo

```yaml
jobs:
  build:
    runs-on: ubuntu-latest

  test:
    runs-on: ubuntu-latest
```

```yaml
jobs:
  my_first_job:
    name: My first job
  my_second_job:
    name: My second job
```

# GitHub Actions [Sintaxe]

- Jobs
  - runs-on
    - Define o tipo de máquina que será executado
  - if
    - Condicional de execução

```yaml
on:
  workflow_dispatch:
    inputs:
      chosen-os:
        required: true
        type: choice
        options:
        - Ubuntu
        - macOS

jobs:
  test:
    runs-on: [self-hosted, "${{ inputs.chosen-os }}"]
    steps:
    - run: echo Hello world!
```

# GitHub Actions [Sintaxe]

- Jobs
  - runs-on
    - Define o tipo de máquina que será executado
  - if
    - Condicional de execução

```yaml
name: example-workflow
on: [push]
jobs:
  production-deploy:
    if: github.repository == 'octo-org/octo-repo-prod'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v4
        with:
          node-version: '14'
      - run: npm install -g bats
```

# GitHub Actions [Sintaxe]

- Jobs
  - environment
  - Settings → Environments
    - dev
    - staging
    - production
  - Cada environment pode ter:
    - secrets próprios
    - aprovação manual
    - regras de proteção



# GitHub Actions [Sintaxe]

- Jobs
  - default
    - Serve para definir configurações padrão para todos os run steps, para você não precisar repetir as mesmas opções várias vezes
      - shell padrão
      - diretório padrão
      - configurações repetidas
      - Usado em workflow e jobs

```yaml
name: teste

on: push

defaults:
  run:
    shell: bash
    working-directory: ./scripts

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - run: ls
      - run: pwd
```

# GitHub Actions [Sintaxe]

- Jobs
  - Concurrency
    - Controlar quantos workflows ou jobs podem rodar ao mesmo tempo
  - Ele é usado para evitar problemas como:
    - dois deploys ao mesmo tempo
    - vários builds iguais rodando juntos
    - sobrescrever produção
    - gastar runner à toa



# GitHub Actions [Sintaxe]

- Steps
  - Executam comandos, tarefas de configuração ou ações em seu repositório
  - Nem todas as etapas executam ações, mas todas as ações são executadas como uma etapa

```yaml
name: Greeting from Mona

on: push

jobs:
  my-job:
    name: My Job
    runs-on: ubuntu-latest
    steps:
      - name: Print a greeting
        env:
          MY_VAR: Hi there! My name is
          FIRST_NAME: Mona
          MIDDLE_NAME: The
          LAST_NAME: Octocat
        run: |
          echo $MY_VAR $FIRST_NAME $MIDDLE_NAME $LAST_NAME.
```

# GitHub Actions [Sintaxe]

- Steps
  - if
    - Adicionar condicional de verificação para execução

Vários comandos podem ser utilizados em diferentes contextos, como workflow, jobs ou steps. A ordem de prioridade é sempre pelo mais específico

```yaml
steps:
  - name: My first step
    if: ${{ github.event_name == 'pull_request' && github.event.action == 'unassigned' }}
    run: echo This event is a pull request that had an assignee removed.
```

# GitHub Actions [Sintaxe]

- Steps
  - run
    - Executa comando shell
  - uses
    - Executa uma action pronta
    - action@heroku1.0.0
      - https://github.com/marketplace?type=actions




Fluxo alternativo: success(), failure(), always() e cancelled()

# GitHub Actions [Sintaxe]

- Steps
  - run
    - Executa comando shell
  - uses
    - Executa uma action pronta
    - action@heroku1.0.0
      - https://github.com/marketplace?type=actions

```
|-- hello-world (repository)
|   |__ .github
|       └── workflows
|           └── my-first-workflow.yml
|       └── actions
|           |__ hello-world-action
|               └── action.yml
```

```yaml
name: Ola Action

inputs:
  nome:
    required: true

runs:
  using: composite
  steps:
    - run: echo "Oi ${{ inputs.nome }}"
      shell: bash
```

# GitHub Actions [Sintaxe]

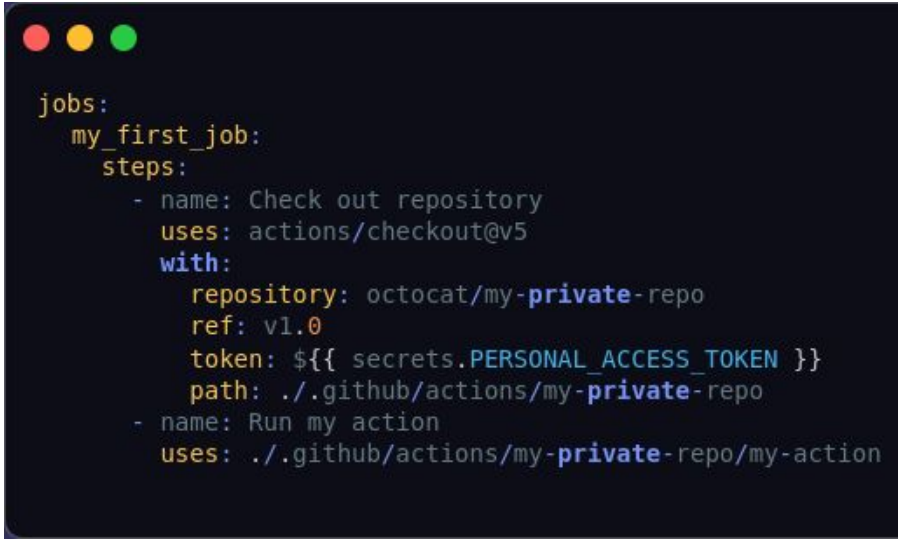
- Steps
  - run
    - Executa comando shell
  - uses
    - Executa uma action pronta
    - action@heroku1.0.0
      - https://github.com/marketplace?type=actions

```
|-- hello-world (repository)
|   |__ .github
|       └── workflows
|           └── my-first-workflow.yml
|       └── actions
|           |__ hello-world-action
|               └── action.yml
```

```yaml
jobs:
  my_first_job:
    runs-on: ubuntu-latest
    steps:
      # This step checks out a copy of your repository
      - name: My first step - check out repository
        uses: actions/checkout@v5
      # This step references the directory that contai
      - name: Use local hello-world-action
        uses: ./.github/actions/hello-world-action
```

# GitHub Actions [Sintaxe]

- Steps
  - uses
    - with
      - Passar parâmetro para uma action que será invocada



```yaml
jobs:
  my_first_job:
    steps:
      - name: Check out repository
        uses: actions/checkout@v5
        with:
          repository: octocat/my-private-repo
          ref: v1.0
          token: ${{ secrets.PERSONAL_ACCESS_TOKEN }}
          path: ./.github/actions/my-private-repo
      - name: Run my action
        uses: ./.github/actions/my-private-repo/my-action
```

# GitHub Actions [Sintaxe]

- Steps
  - uses
    - with
      - Passar parâmetro para uma action que será invocada
  - input
    - Parâmetros a serem recebidos em uma action

```yaml
name: Minha Action com parametros

description: Action de teste

inputs:
  nome:
    description: Nome da pessoa
    required: true

  idade:
    description: Idade da pessoa
    required: false
    default: 0

runs:
  using: composite

  steps:
    - run: echo "Nome = ${{ inputs.nome }}"
      shell: bash
```

# GitHub Actions [Sintaxe]

- Steps
  - working-directory
    - Serve para definir em qual pasta o comando run será executado

```yaml
steps:
  - run: npm install
    working-directory: ./backend

  - run: npm run build
    working-directory: ./frontend
```

```yaml
steps:
  - run: npm install
    working-directory: ./backend

  - run: npm test
    working-directory: ./backend
```



```yaml
jobs:
  build:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ./backend

    steps:
      - run: npm install
      - run: npm test
```

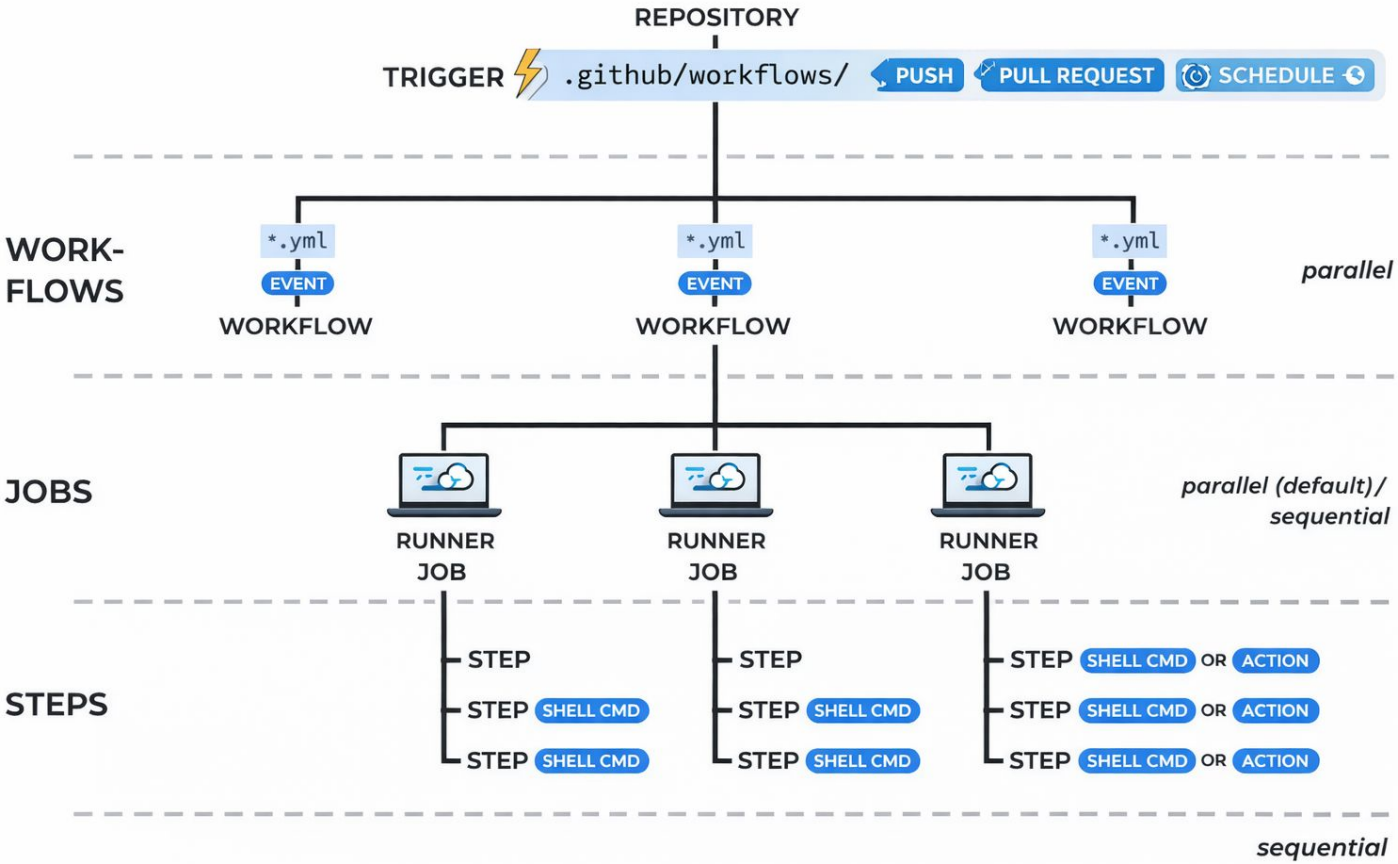
# GitHub Actions [Sintaxe]

- Steps
  - shell
    - Serve para definir qual interpretador de comando será usado para executar o run
    - bach, cmd, powershell, python, node …

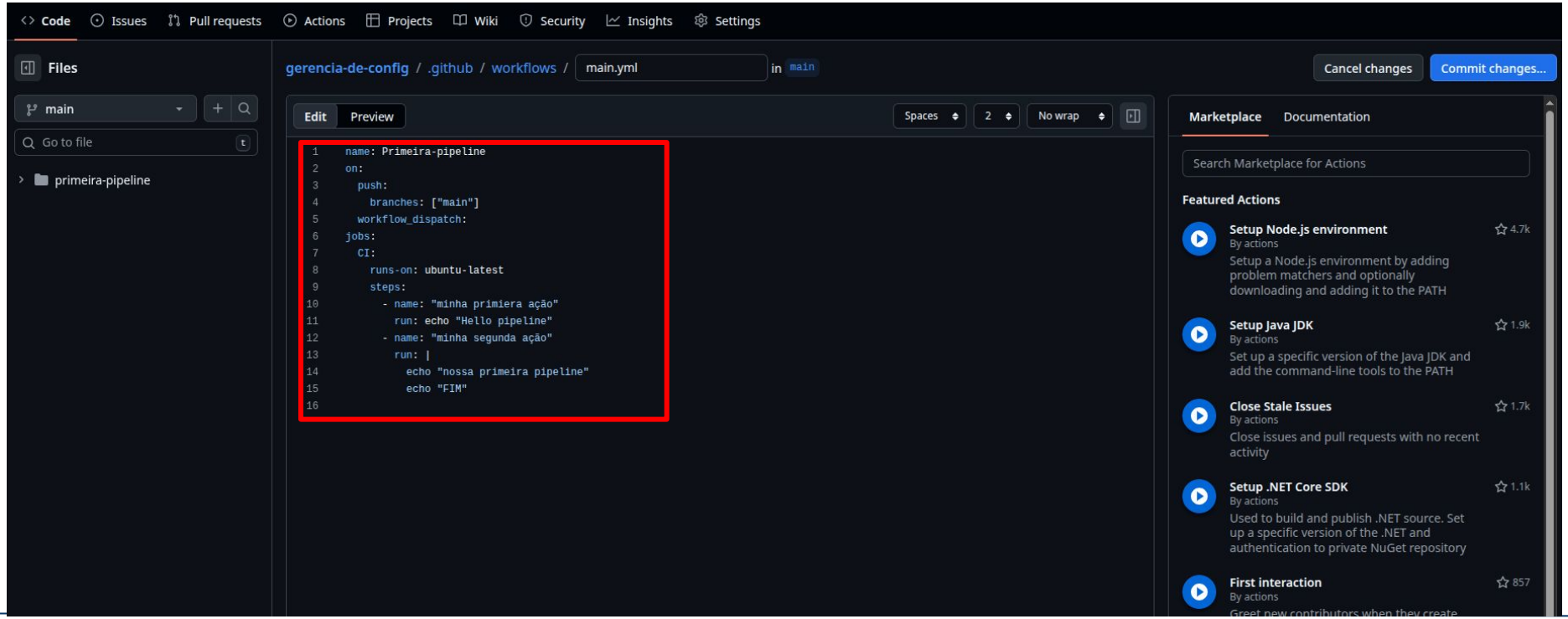
```yaml
runs-on: windows-latest

steps:
  - run: |
      echo "Hello from bash"
      ls
    shell: bash
```

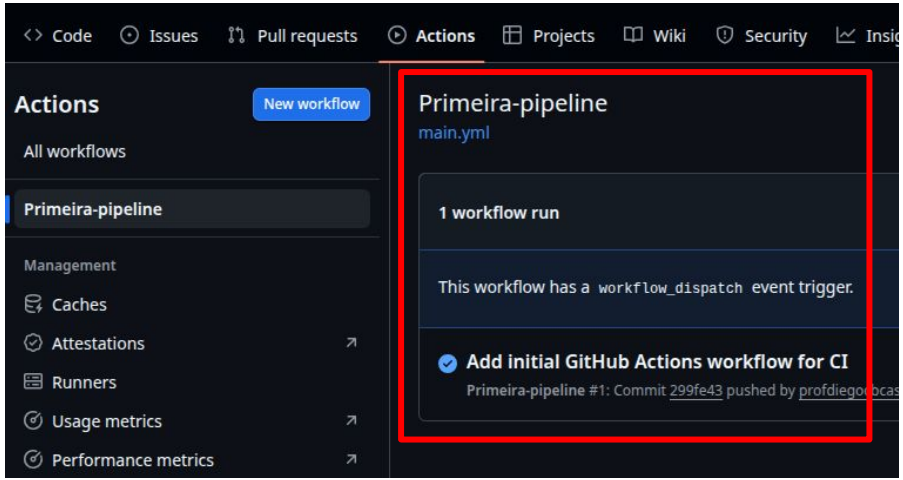
- jobs.<job_id>.steps
- jobs.<job_id>.steps[*].id
- jobs.<job_id>.steps[*].if
- jobs.<job_id>.steps[*].name
- jobs.<job_id>.steps[*].uses
- jobs.<job_id>.steps[*].run
- jobs.<job_id>.steps[*].working-directory
- jobs.<job_id>.steps[*].shell
- jobs.<job_id>.steps[*].with
- jobs.<job_id>.steps[*].with.args
- jobs.<job_id>.steps[*].with.entrypoint
- jobs.<job_id>.steps[*].env
- jobs.<job_id>.steps[*].continue-on-error
- jobs.<job_id>.steps[*].timeout-minutes

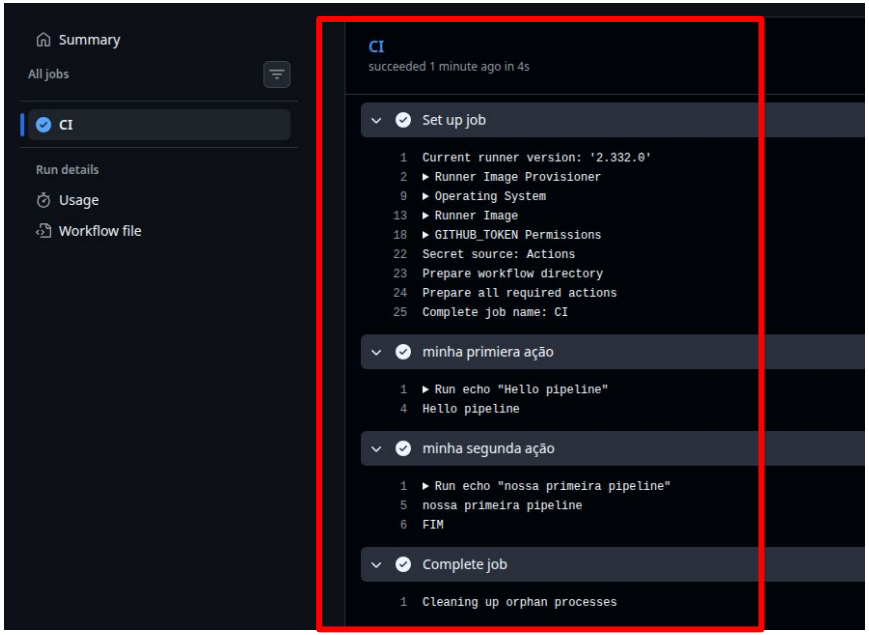

REPOSITORY
TRIGGER
.github/workflows/
PUSH
PULL REQUEST
SCHEDULE
WORK-FLOWS
*.yml
EVENT
WORKFLOW
*.yml
EVENT
WORKFLOW
*.yml
EVENT
WORKFLOW
parallel
JOBS
RUNNER JOB
RUNNER JOB
RUNNER JOB
parallel (default)/ sequential
STEPS
STEP
STEP SHELL CMD
STEP SHELL CMD
STEP
STEP SHELL CMD
STEP SHELL CMD
STEP SHELL CMD OR ACTION
STEP SHELL CMD OR ACTION
STEP SHELL CMD OR ACTION
sequential

# GitHub Actions



# GitHub Actions



# GitHub Actions [Container]

- Serve para fazer o job rodar dentro de um container Docker, em vez de rodar diretamente na máquina do runner
- Isso é muito usado quando você quer:
  - garantir ambiente igual
  - usar uma imagem específica
  - rodar CI com Docker

```yaml
name: CI
on:
  push:
    branches: [ main ]
jobs:
  container-test-job:
    runs-on: ubuntu-latest
    container:
      image: node:18
      env:
        NODE_ENV: development
      ports:
        - 80
      volumes:
        - my_docker_volume:/volume_mount
      options: --cpus 1
    steps:
      - name: Check for dockerenv file
        run: (ls /.dockerenv && echo Found dockerenv) || (echo No dockerenv)
```

# GitHub Actions [Container]

- Services
  - São containers extras que rodam junto com o job, normalmente para fornecer dependências como banco de dados, cache, fila, etc

```yaml
name: CI Node + Jest + Postgres

on:
  push:
  pull_request:

jobs:
  test:

    runs-on: ubuntu-latest

    # container onde o job roda
    container:
      image: node:20

    # services extras
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: testdb
        ports:
          - 5432:5432

    env:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: test
      DB_PASS: test
      DB_NAME: testdb
```

```yaml
steps:

  - name: Checkout
    uses: actions/checkout@v4

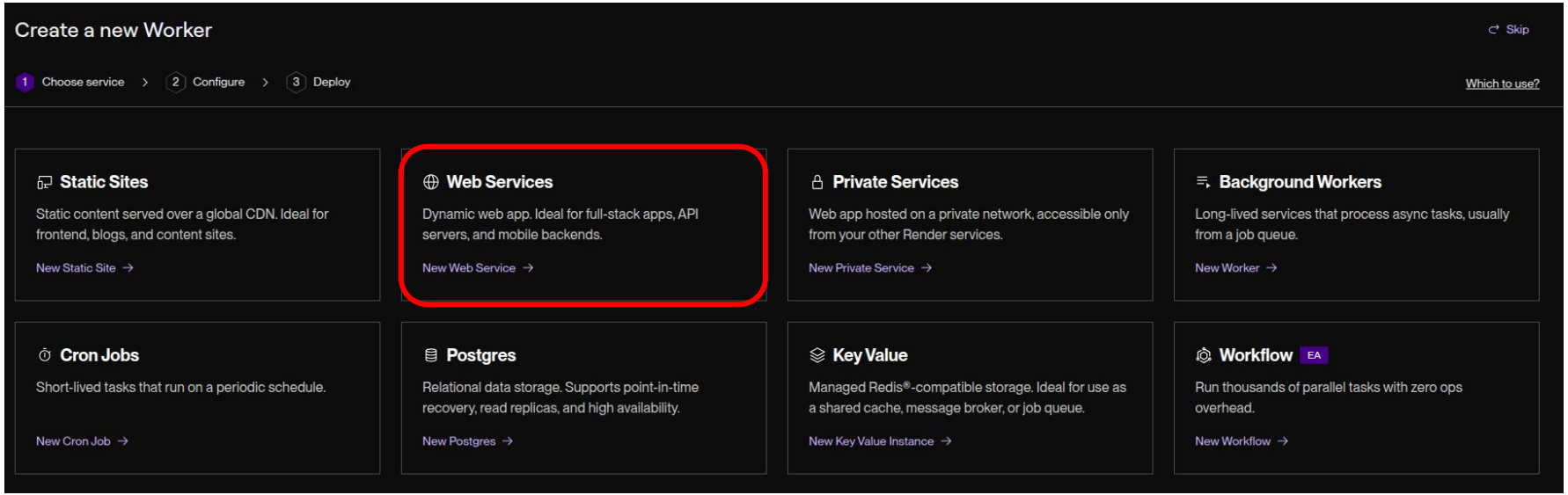
  - name: Install deps
    run: npm install

  - name: Run tests
    run: npm test
```

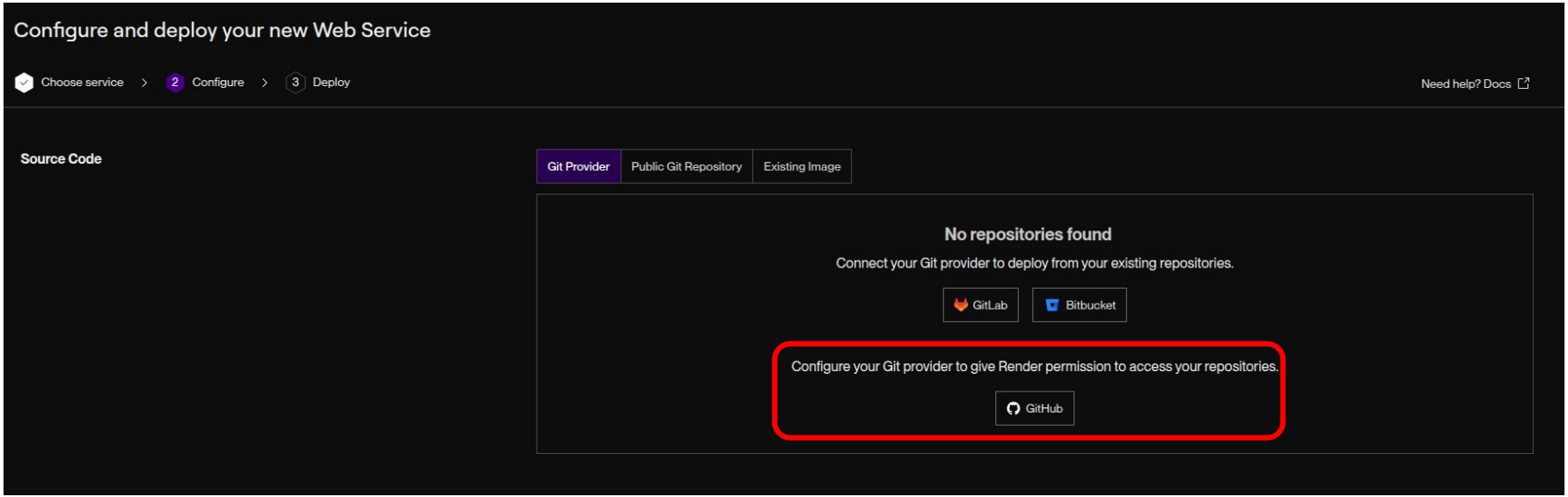
# GitHub Actions

- Clouds free
  - Render (tem free tier e aceita Docker)
    - https://render.com
  - Railway (free com limite mensal)
  - Fly.io (bom para containers)
  - Vercel (melhor para frontend / serverless)

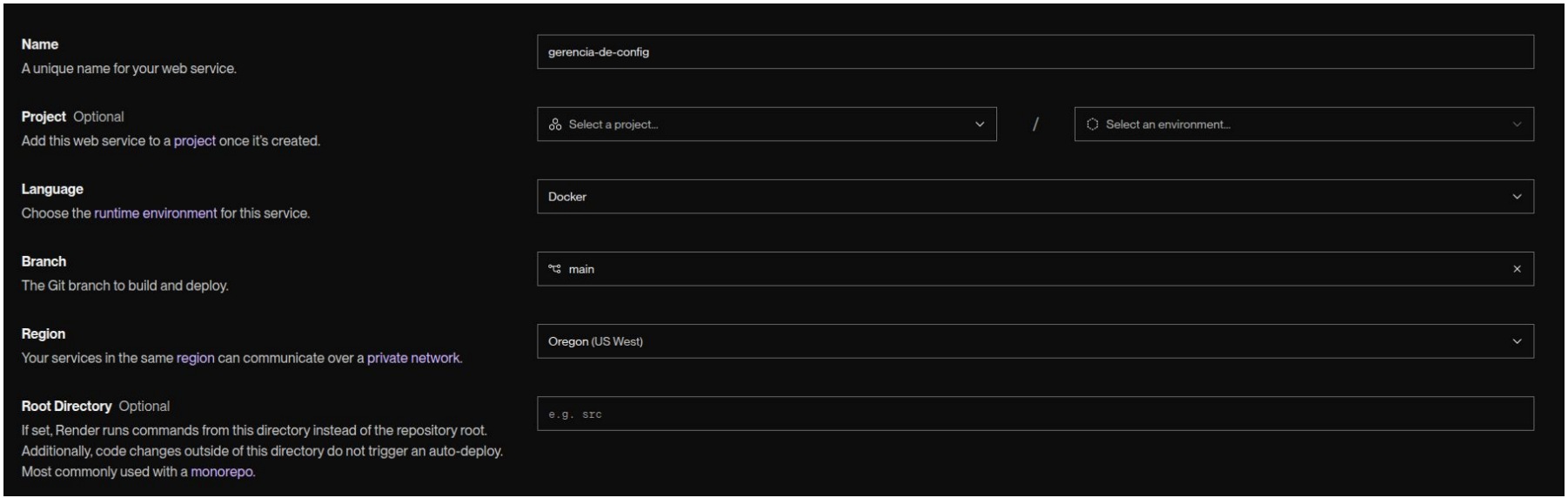
# GitHub Actions



# GitHub Actions



# GitHub Actions



# GitHub Actions




Vai dar erro!

```yaml
name: Deploy Node API

on:
  push:
    branches: [ main ]

jobs:

  build:

    runs-on: ubuntu-latest

    steps:

      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install deps
        run: npm install

      - name: Test build
        run: node index.js &

      - name: Wait server
        run: sleep 5

  deploy:

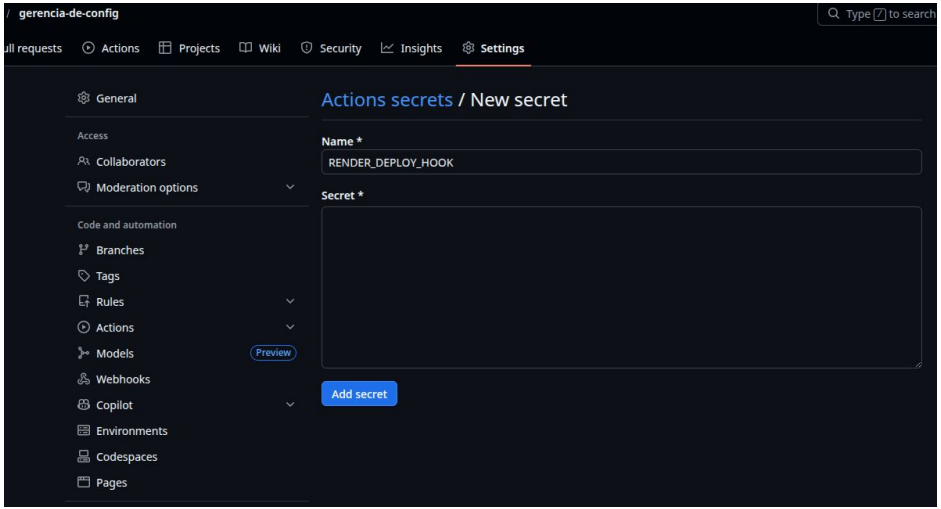
    needs: build

    runs-on: ubuntu-latest

    steps:

      - name: Deploy Render
        run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```

# GitHub Actions







```yaml
name: Deploy Node API

on:
  push:
    branches: [ main ]

jobs:

  build:

    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: primeira-pipeline

    steps:

      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20

      - run: npm install

  deploy:

    needs: build

    runs-on: ubuntu-latest

    steps:

      - name: Deploy Render
        run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```